

Deep Research Report for the User's Project

Executive Summary

The uploaded capstone materials indicate that the project is intended to be a **multimodal emotion recognition system** built over **audio, video, and text**, with parallel modality-specific pipelines and a fusion stage. However, several project-defining details are still unknown from the available materials: the exact benchmark dataset, the final emotion taxonomy, the target deployment language, the latency requirement, the compute budget, and the production environment. I therefore treat this as a **partially specified multimodal classification project** and make the missing assumptions explicit below rather than silently guessing.

Given that framing, I recommend a **two-stage delivery strategy**. First, build reproducible unimodal baselines and a **calibrated late-fusion system**; then, if compute and time allow, add a stronger **cross-modal fusion model** such as **MuT**, **MAG**, or **MISA**. For the headline metric, I recommend **macro-F1**. For secondary metrics, I recommend **weighted-F1, accuracy, per-class precision/recall, confusion matrices, Brier score, calibration curves, and latency/throughput**. That metric package best reflects multi-class emotion recognition quality while also forcing the system to be deployment-aware. ¹

For data, the most defensible benchmark stack is **MELD** for multi-party conversational emotion recognition, **IEMOCAP** for acted dyadic multimodal emotion data, and **CMU-MOSEI** for broader, in-the-wild multimodal generalization. MELD contains about 13,000 utterances from 1,433 dialogues; IEMOCAP releases about 12 hours of audiovisual data with aligned transcriptions; CMU-MOSEI provides more than 23,000 annotated sentence-level segments from more than 1,000 speakers and 250 topics. The **CMU Multimodal SDK** is especially useful if you need alignment and modality handling in a reproducible research pipeline. ²

If the eventual product language is Turkish, I would begin with **BERTurk** for text and benchmark ASR from OpenAI ³ carefully rather than assuming transcript quality. Whisper's official release highlights strong multilingual robustness, but its model card also warns that hallucinations and other weaknesses may be worse in lower-resource or lower-discoverability languages. That matters directly if Turkish transcripts feed the classifier. ⁴

For deployment, the simplest path is **PyTorch training → ONNX Runtime inference**. If the system evolves into multi-model serving with concurrent requests, **Triton Inference Server** becomes more attractive, and if the target is Kubernetes-native production, **KServe** adds autoscaling, revision tracking, and canary/A/B rollout support. For cloud benchmarking, public listings show **g4dn.xlarge** starting around **\$0.526/hour** and **g5.xlarge** around **\$1.006/hour** in us-east-1; that is enough to build realistic minimal, recommended, and extensive compute profiles. ⁵

What the Available Materials Establish

The uploaded capstone proposal and final-report templates consistently describe a project centered on **multimodal emotion recognition using audio, video, and text**. They also indicate an intended architecture with separate modality pipelines and a fusion stage, plus a 14-week academic delivery horizon. At the same time, the materials do **not** lock down the most important engineering decisions I

would need for a full implementation audit: the final dataset selection, the exact emotion labels, the split protocol, whether transcripts are oracle or ASR-generated, whether the target is English-only or multilingual, and whether the deployment goal is offline benchmarking or real-time inference.

That distinction matters. The uploaded materials are strong enough to define the **problem class**, but not strong enough to define a single unambiguous **production system specification**. I therefore treat the report below as a rigorous **design-and-research blueprint** for the project the materials imply, not as a claim that a specific benchmark already exists or that any placeholder result table in the templates has been independently verified.

The key unknowns I am carrying as explicit assumptions are below.

Unknown detail	Why it matters	Working assumption I use in this report
Primary benchmark dataset	Changes preprocessing, split policy, and expected performance	Start with MELD or IEMOCAP; use CMU-MOSEI as a secondary generalization benchmark
Emotion taxonomy	Changes loss design and comparability across datasets	Treat the main task as discrete multi-class emotion classification; if multiple datasets are combined, map to a shared reduced label space
Language target	Determines whether English-only models are acceptable	Assume English benchmark research first; support Turkish as a product extension path
Transcript source	ASR quality can dominate final performance	Separate oracle-transcript and ASR-transcript experiments
Real-time requirement	Drives architecture and serving choices	Assume offline/batch research first; near-real-time serving as a stretch goal
Deployment environment	Changes serving stack and optimization priorities	Assume a single-GPU prototype first, then optional Kubernetes serving
Compute budget	Determines whether video transformers are realistic	Assume one modest GPU for baseline work and one stronger single GPU for recommended training
Missing-modality behavior	Crucial in real use, especially for face or ASR failures	Design for degraded but functional inference if one modality is weak or absent
Data governance rules	Critical if any new user data is collected	Assume public datasets for research; if new data is collected, explicit consent and retention controls are required

Goals, Success Metrics, and Project Framing

Because the project is not fully specified, the common goal options are worth naming explicitly. In practice, projects like this usually optimize one or more of five things: **discrete emotion classification**, **continuous affect regression** such as valence/arousal/dominance, **robustness to missing or noisy modalities**, **near-real-time inference**, and **language/domain portability**. MELD naturally supports conversational discrete emotion classification; CMU-MOSEI also opens the door to continuous affect

targets. I recommend treating **multi-class emotion classification** as the primary academic objective and **missing-modality robustness** plus **deployment readiness** as engineering objectives. ⁶

I recommend the following evaluation package. The logic is straightforward: a multimodal emotion system should not be judged by a single scalar that can be dominated by common classes or overconfident mistakes. The **primary metric should be macro-F1** because F1 explicitly combines precision and recall, and macro averaging prevents majority classes from carrying the entire headline result. Secondary metrics should include weighted-F1 and accuracy for comparability, per-class precision/recall for error diagnosis, confusion matrices for emotion-pair collapse analysis, and probability calibration metrics because fusion systems often produce impressive-looking—but poorly calibrated—confidence scores. ⁷

Metric role	Metric	Why I recommend it
Primary	Macro-F1	Best headline metric for multi-class emotion tasks when class distributions are uneven
Secondary	Weighted-F1	Preserves comparability with class-frequency-weighted performance
Secondary	Accuracy	Useful headline number, but insufficient on its own
Secondary	Per-class precision and recall	Exposes minority-class failure modes and asymmetric errors
Secondary	Confusion matrix	Shows which emotions are being confused with each other
Secondary	Brier score and calibration curve	Measures confidence quality, not just correctness
Secondary	Latency, throughput, missing-modality rate	Ensures the system is operable, not just accurate in a notebook

If Turkish becomes a hard requirement, I would keep the evaluation package unchanged but add one more validation axis: **language-conditioned evaluation**. That means reporting scores separately for English benchmark data and any Turkish target data rather than collapsing them into one blended metric. For Turkish text, BERTurk is a sensible starting point; for Turkish speech-to-text, I would treat ASR noise as a first-class experimental variable rather than background infrastructure. ⁸

Data, Preprocessing, and Features

The project needs a much broader conception of “data” than just audio clips and frames. A robust multimodal system depends on **raw media**, **aligned time series**, **structured metadata**, **labels**, and **quality indicators**. The most useful public sources for this project are MELD, IEMOCAP from University of Southern California ⁹, and the CMU-MOSEI ecosystem from Carnegie Mellon University ¹⁰. The SDK around the CMU datasets is particularly valuable because it simplifies multimodal loading and alignment. ¹¹

Data type	What it means in this project	Likely sources	Main preprocessing needs
Structured metadata	<code>utterance_id</code> , <code>dialogue_id</code> , <code>speaker_id</code> , timestamps, split assignment	Dataset CSV/ JSON manifests	Integrity checks, duplicate detection, leak- safe split assignment
Unstructured audio	Raw speech waveforms	MELD, IEMOCAP, CMU-MOSEI	Resampling, normalization, VAD/ silence handling, segmentation
Unstructured video	Full clips or utterance-level cut segments	Same datasets	Frame sampling, face detection, face alignment, resize/crop
Images	Detected face frames or keyframes	Generated from video	Face-quality filtering, temporal indexing
Time series	MFCC frames, pitch/energy contours, frame embeddings, aligned multimodal sequences	Derived from raw media or SDK outputs	Synchronization, padding/masking, interpolation
Text	Gold transcripts or ASR transcripts	Dataset transcripts or ASR pipeline	Cleaning, tokenization, speaker-context assembly
Labels	Emotion class, sentiment, optional valence/arousal/ dominance	Dataset annotations	Label normalization, cross-dataset mapping if needed
Quality signals	ASR confidence, face-detection success, missing modality flags	Derived metadata	Used for fallback logic and monitoring

The data collection and preprocessing stack should be built around a single **manifest table** that knows where every modality lives and how it aligns. For audio, I would standardize sampling rate, normalize amplitude, remove or tag silence, and preserve both waveform access and frame-level acoustic features. Librosa provides a direct MFCC pipeline; for speech encoders, the raw waveform should also remain available. For video, I would not begin with dense frame extraction. A lighter **2–4 FPS** strategy is usually enough for a capstone-scale first pass, followed by face detection and cropping with OpenCV’s DNN-based tooling. For text, I would preserve both the current utterance and a small window of conversational context because conversation emotion recognition often benefits from local dialogue history. ¹²

A crucial experimental choice is how transcripts enter the system. If the dataset already includes transcripts, I would run **two separate regimes**: an **oracle transcript regime** to characterize the ceiling of the emotion recognizer itself, and an **ASR regime** to simulate deployment. This separation is especially important because Whisper’s strengths are real, but its official materials also acknowledge hallucination and repetition risks, which can be worse in lower-resource settings. This is why I would store ASR confidence and transcript provenance as structured features rather than treating transcripts as always-correct text. ¹³

Feature engineering should not be split into a false choice between handcrafted and learned features. In this project, the strongest practical design is usually **learned modality embeddings plus a small set of handcrafted reliability features**. Handcrafted audio features remain useful for cheap baselines and fallback models; handcrafted visual features can expose face-quality instability; and a few text-side indicators can capture context breaks that pretrained encoders do not explicitly model. Learned features then carry the heavy semantic and acoustic burden. ¹⁴

Feature family	Handcrafted examples	Learned examples	Where it helps most
Audio	MFCC, pitch, energy, speaking-rate proxies, pause ratio	wav2vec 2.0, HuBERT embeddings	Strong baselines, noisy-speech robustness, fallback models
Video	Face box size stability, landmark motion statistics, head pose	EfficientNet feature maps, TimeSformer frame-patch embeddings	Expression dynamics, visual confidence estimation
Text	Length, punctuation, speaker-change markers, context-turn count	BERT, BERTurk contextual embeddings	Semantic emotion cues, sarcasm/context handling
Fusion metadata	Missing-modality flags, ASR confidence, face-detection confidence	Learned gating weights	Better fusion and safer degraded inference

Model Landscape and Recommended Architecture

The model landscape for this project spans **classical ML, sequence models, CNNs, transformers, GNNs, ensemble methods**, and, more cautiously, **diffusion-based augmentation**. I do not recommend attacking the problem with the most sophisticated architecture first. I recommend starting from the smallest model family that answers the current research question. If you first need a trustworthy baseline, classical methods over engineered or frozen features are excellent. If you need state-of-the-art single-modality backbones, self-supervised speech and transformer text models are much stronger. If you need explicit multi-party conversational structure, a graph formulation becomes more attractive. ¹⁵

The comparison below is my synthesis of the cited papers and official tool documentation. The **expected performance** column is intentionally **relative**, not a promise of benchmark numbers.

Candidate model	Use-case fit	Data needs	Pros / cons	Compute	Expected performance
SVM / XGBoost / LightGBM	Strong baseline on handcrafted or frozen embeddings; especially good for small datasets or sanity checks	Clean tabular or pooled features	Pros: cheap, interpretable, fast to iterate. Cons: weak at long temporal and cross-modal interaction modeling; RBF SVM scales poorly with dataset size	Low	Low to medium
BiLSTM / GRU over modality features	Good when temporal order matters but compute is limited	Sequential feature vectors per modality	Pros: cheaper than transformers, easy to implement. Cons: weaker long-range modeling, harder to parallelize than attention-based models	Low to medium	Medium
wav2vec 2.0 / HuBERT + classifier	Best first serious audio backbone	Raw audio or segmented waveforms	Pros: strong self-supervised speech representations, robust with limited labels. Cons: still audio-only; GPU needed for fine-tuning	Medium	Medium to high
EfficientNet + temporal pooling	Best capstone-tier visual backbone for face frames	Face crops or keyframes	Pros: efficient, solid transfer learning, easier than video transformers. Cons: can miss richer temporal structure	Medium	Medium to high

Candidate model	Use-case fit	Data needs	Pros / cons	Compute	Expected performance
TimeSformer or similar video transformer	Better when facial dynamics and temporal structure are central	More frames, more stable alignment, more VRAM	Pros: stronger spatiotemporal modeling. Cons: more expensive and more data-hungry	High	High
BERT / BERTurk / multilingual text transformer	Best first serious text backbone	Clean transcripts and optional short context windows	Pros: strong semantic modeling, easy fine-tuning ecosystem. Cons: cannot observe tone or facial expression directly	Low to medium	Medium to high
Late fusion / soft voting	Best first multimodal deliverable	Calibrated unimodal probabilities or embeddings	Pros: robust, observable, easy to debug. Cons: weaker explicit cross-modal interaction learning	Low to medium	High
Stacking meta-learner	Best when unimodal errors are complementary	Out-of-fold predictions and metadata	Pros: often better than simple voting. Cons: leakage risk if stacking is not done correctly	Medium	High
MuT / MAG / MISA	Best research-grade multimodal fusion family	Synchronized or weakly aligned modality embeddings	Pros: models cross-modal interactions directly. Cons: more complex, harder to tune, more data-sensitive	Medium to high	High to very high

Candidate model	Use-case fit	Data needs	Pros / cons	Compute	Expected performance
DialogueGCN or other speaker graph model	Best when dialogue structure and speaker interactions drive emotion	Dialogue turns, speaker graph, conversation structure	Pros: explicitly models inter-speaker dependencies. Cons: only makes sense for conversation data	Medium to high	High on conversation datasets
Diffusion-based augmentation / imputation	Useful only as an auxiliary component	Considerable data and careful control	Pros: may help augmentation or missing-modality recovery. Cons: poor first-line choice for the core classifier; expensive and risky for a capstone	High to very high	Indirect gain only

This synthesis points to a clear recommendation. I would make the **first champion system** a three-branch pipeline: **speech encoder + text transformer + lightweight video CNN**, followed by **late fusion** and then optional **stacking**. Only after that system is stable would I move to **cross-modal transformer fusion**. In other words, I would deliberately optimize for **debuggability first, interaction modeling second**. That sequencing is what most often turns a capstone into a reproducible research result instead of a fragile demo. ¹⁶

The hyperparameter ranges below are not meant as “ground truth.” They are the search ranges I recommend for a capstone-scale project built on the cited model families and expected single-GPU constraints.

Component	Recommended starting point	Search range	Notes
Classical baseline	XGBoost or LightGBM on pooled multimodal features	<code>n_estimators</code> 200–800, <code>max_depth</code> 4–10, <code>lr</code> 0.03–0.15, <code>subsample</code> 0.6–1.0	Excellent for fast ablations and metadata-heavy fusion
SVM baseline	Linear or RBF SVM on pooled features	<code>C</code> 0.1–10, <code>gamma</code> scale/auto, kernel <code>linear</code> or <code>rbf</code>	Keep for small-data sanity checks only
Audio encoder	wav2vec 2.0 base or HuBERT base	LR <code>1e-5</code> – <code>5e-5</code> , batch <code>8</code> – <code>32</code> , segment <code>4</code> – <code>8</code> s, dropout <code>0.1</code> – <code>0.3</code>	Freeze lower layers for early epochs if data is limited

Component	Recommended starting point	Search range	Notes
Video CNN	EfficientNet-B0/B2 + attentive temporal pooling	LR $1e-4$ – $3e-4$, batch 8 – 32 , FPS 2 – 4 , frames 8 – 32	Strongest cost/performance default for visual branch
Video transformer	TimeSformer-small style setup	LR $1e-5$ – $3e-5$, batch 2 – 8 , FPS 2 – 4 , frames 8 – 16	Use only if GPU budget is comfortable
Text encoder	BERT base or BERTurk	LR $1e-5$ – $3e-5$, batch 16 – 64 , max length 64 – 256	Use short dialogue context, not full-history by default
Fusion MLP	Concatenate unimodal embeddings + 2-layer MLP	hidden 256 – 1024 , dropout 0.1 – 0.4	Best first production-grade multimodal model
Cross-modal transformer	2–4 fusion layers, 4–8 attention heads	hidden 256 – 768 , heads 4 – 8 , layers 2 – 4 , dropout 0.1 – 0.4	Candidate upgrade after late fusion baseline
Optimization	AdamW	weight decay 0.01 , warmup 5% – 10% , epochs 10 – 30 , early stopping 3 – 5	Add class weighting if labels are skewed
Robustness regularization	Modality dropout + mixup/augmentation where safe	modality dropout 0.1 – 0.3	Important if face detection or ASR is brittle

The combination strategy should also be staged. I recommend **soft voting first**, because it is simple and transparent; **stacking second**, because it can learn complementarity among unimodal branches; and **attention-/gating-based multimodal fusion third**, when the pipeline is stable enough to justify more complexity. **RAG** is not a core emotion-classification method here, but it can be valuable for **retrieval-grounded explanations, project-document Q&A, or evidence-aware demo layers**. **Diffusion** should be considered only as an auxiliary augmentation or imputation tool, not the primary classifier. ¹⁷

The end-to-end architecture I recommend is below. It matches the uploaded project direction, but makes the missing engineering decisions explicit and production-oriented. The design is grounded in multimodal datasets and alignment tooling, pretrained encoders, and pragmatic serving infrastructure.

¹⁸

flowchart LR

A[Raw multimodal data
MELD / IEMOCAP / CMU-MOSEI] --> B[Manifest and split registry
metadata + leak-safe grouping]

B --> C1[Audio preprocessing
16 kHz resample, normalize, VAD, segment]

B --> C2[Video preprocessing
frame sampling, face detect, align,

```

crop]
  B --> C3[Text preprocessing<br/>gold transcript or ASR, tokenize, add
context]

  C1 --> D1[Audio encoder<br/>wav2vec2 / HuBERT]
  C2 --> D2[Video encoder<br/>EfficientNet / TimeSformer]
  C3 --> D3[Text encoder<br/>BERT / BERTurk]

  D1 --> E[Fusion layer<br/>late fusion -> stacking -> cross-modal
transformer]
  D2 --> E
  D3 --> E

  E --> F[Classifier<br/>emotion logits and probabilities]
  F --> G[Calibration<br/>temperature scaling / reliability checks]
  G --> H[Packaging<br/>ONNX Runtime / Triton / KServe]

  B --> I[Experiment lineage<br/>DVC + MLflow]
  F --> I
  H --> J[Monitoring<br/>latency, drift, confidence, missing modality]

```

Training, Validation, and Evaluation

I would build the training system as a **versioned DAG**, not as a monolithic notebook. A clean pipeline should have at least these stages: data manifest creation, modality preprocessing, cached feature generation where appropriate, unimodal training, multimodal fusion training, evaluation, and packaging. **DVC** is well suited to represent the pipeline as a dependency graph and reproduce it end to end; **MLflow** is well suited to track experiments, models, and lineage; **PyTorch** is the natural core framework; **PyTorch Lightning** reduces repetitive training boilerplate; **SpeechBrain** is useful for speech recipes; and the pretrained-model ecosystem from Hugging Face ¹⁹ makes encoder experimentation much faster. I would also log seeds, hashes, configs, and deterministic settings as far as feasible, because PyTorch explicitly documents sources of nondeterminism and the steps needed to limit them.

²⁰

The validation protocol must be leakage-aware from the beginning. Emotion datasets are particularly vulnerable to **speaker leakage**, **dialogue leakage**, and **session leakage**, which can inflate scores dramatically while teaching the wrong lesson. If you work on MELD-like conversation data, the split should be grouped by **dialogue**; if you work on IEMOCAP-like data, grouping by **session or speaker pair** is safer. When class balance matters and the number of groups is large enough, **StratifiedGroupKFold** is the right default. When strict grouping is more important than class-balance preservation, **GroupKFold** is safer. If you stack models, the meta-learner should only see **out-of-fold predictions**, not in-fold predictions. ²¹

The evaluation protocol should report more than the final test number. I would require: a validation-set model selection record, at least one repeated-seed run for the recommended system, unimodal ablations, modality-drop ablations, oracle-vs-ASR transcript comparison, and cross-dataset evaluation when possible. The final report should include macro-F1, weighted-F1, accuracy, per-class metrics, a confusion matrix, and confidence calibration diagnostics. Scikit-learn's evaluation, calibration, confusion matrix, and Brier score tools are sufficient for this if used consistently. ⁷

I also recommend one additional evaluation lens that is often skipped in student projects: **operational failure analysis**. That means explicitly measuring what happens when one modality is absent or degraded. In practice, I would report at least four stress tests: audio-noise corruption, face-detection failure, transcript-ASR degradation, and single-modality dropout at inference. A multimodal model that collapses under any one of those conditions is not yet robust, no matter how good its clean validation score looks.

Deployment, Monitoring, Compute, Risks, and Timeline

For deployment, I recommend choosing the serving stack based on actual operational scope, not perceived sophistication. If the target is a single packaged prototype, **ONNX Runtime** is the simplest credible path because it is cross-platform and accelerates models from multiple upstream frameworks. If the system becomes a **multi-branch inference graph** with multiple external models and concurrent traffic, **Triton Inference Server** is more compelling because it supports multi-framework serving, HTTP/gRPC endpoints, model instances, and dynamic batching. If the target becomes a managed Kubernetes service, **KServe** adds autoscaling, health checking, revision tracking, canary rollout, and A/B testing. ²²

For runtime optimization, the first two knobs I would reach for are **ONNX quantization** and **Triton dynamic batching**. ONNX Runtime documents 8-bit linear quantization as a core optimization path, and Triton explicitly documents dynamic batching as a major throughput optimization for stateless models. Those are practical, incremental optimizations that fit a capstone much better than premature micro-optimization of every branch. ²³

Monitoring should cover both systems and models. At the infrastructure level: **latency, throughput, memory, GPU utilization, and failure rate**. At the model level: **class distribution drift, confidence drift, missing-modality rate, face-detection success rate, ASR confidence, and calibration drift**. For this class of system, I would also log per-request modality availability because it is often the hidden root cause of real-world quality drops.

The compute and cost table below is deliberately conservative. It assumes public **on-demand** reference prices rather than spot pricing, and it uses three profiles: a minimum viable research profile, a recommended single-GPU profile, and an extensive profile for ablations and more ambitious fusion models. AWS documents G4dn as cost-effective for ML inference and G5 as materially stronger for training; public pricing aggregators list g4dn.xlarge at about **\$0.526/hour** and g5.xlarge at about **\$1.006/hour** in us-east-1. Exact pricing varies by region and date, so I treat these as planning numbers, not procurement quotes. ²⁴

Profile	Objective	Instance assumption	Estimated GPU-hours	Approx compute-only cost	What fits in scope
Minimal	Sanity baselines and one simple multimodal fusion model	g4dn.xlarge	60–120	about \$32–\$63	Handcrafted features, frozen encoders, small fine-tunes

Profile	Objective	Instance assumption	Estimated GPU-hours	Approx compute-only cost	What fits in scope
Recommended	Full capstone-grade system with pretrained audio/text and light video branch	g5.xlarge	120-240	about \$121-\$241	wav2vec/ HuBERT, BERT/ BERTurk, EfficientNet branch, late fusion, stacking
Extensive	Ablations, repeated seeds, stronger fusion, broader robustness testing	g5.xlarge-equivalent	400-800	about \$402-\$805	Cross-modal transformer fusion, more dataset variants, more repeats

The main risks and mitigations are synthesized below. They are the failure modes I would expect to dominate project outcome more than raw model choice.

Risk	Why it matters	Mitigation I recommend
Speaker/dialogue leakage	Inflates validation scores and misleads model selection	Use GroupKFold or StratifiedGroupKFold with dialogue/session grouping
ASR errors and hallucinations	Pollutes the text branch and can dominate fusion errors	Separate oracle and ASR experiments; track transcript provenance and ASR confidence
Class imbalance	Makes accuracy look better than actual class coverage	Optimize and report macro-F1, class weights, per-class recall
Modality desynchronization	Breaks early fusion and weakens temporal interaction models	Build a timestamped manifest and validate alignment at preprocessing time
Visual failure cases	Occlusion, lighting, and missed faces can silently break the video branch	Store face-detection confidence and support missing-modality inference
Domain and language shift	Public datasets do not guarantee real-world transfer, especially into Turkish	Run cross-dataset or held-out-domain tests; keep Turkish evaluation separate
Compute overrun	Video-heavy experiments expand cost and iteration time quickly	Cache derived artifacts, use moderate FPS, start with EfficientNet not video transformers
Privacy and consent	Speech and facial data are sensitive if new data is collected	Stay on public datasets unless there is a clear governance plan

The uploaded proposal already implies a **14-week** delivery horizon. I would refine it into the following milestone plan.

Weeks	Milestone	Deliverable
1–2	Scope lock and data audit	Dataset decision, label policy, split policy, experiment registry design
3–4	Preprocessing pipeline	Audio/video/text preprocessing scripts, manifest schema, leak-safe split implementation
5–6	Unimodal baselines	Audio-only, video-only, text-only baselines with first error analysis
7–8	Strong pretrained branches	wav2vec/HuBERT, BERT/BERTurk, EfficientNet branch stabilized
9–10	Multimodal fusion	Late fusion, calibration, stacking, ablation against unimodal models
11–12	Research upgrade	MuT/MAG/MISA or DialogueGCN-style experiment plus robustness tests
13	Serving prototype	ONNX Runtime or Triton prototype with logging and monitoring hooks
14	Final packaging	Final report, tables, diagrams, reproducibility appendix, presentation

Selected Primary Resources

The papers I would treat as core reading for this project are **MELD**, **IEMOCAP**, **CMU-MOSEI**, **Attention Is All You Need**, **BERT**, **wav2vec 2.0**, **HuBERT**, **MuT**, **MAG**, **MISA**, **DialogueGCN**, and the original **RAG** paper. Together, they cover the benchmark datasets, the dominant unimodal backbones, the main multimodal fusion paradigms, and the auxiliary retrieval pattern that can support explanations or knowledge-grounded demo layers. ²⁵

For implementation, the library and systems stack I would prioritize is **Transformers**, **PyTorch**, **ONNX Runtime**, **Triton Inference Server**, **DVC**, **MLflow**, **Whisper**, **BERTurk**, **SpeechBrain**, **Diffusers**, plus the supporting signal-processing libraries **librosa** and **OpenCV**. Those tools jointly cover pretrained modeling, training, reproducibility, inference acceleration, serving, ASR, Turkish text support, and optional generative augmentation. ²⁶

My bottom-line recommendation is simple: **build the reproducible late-fusion baseline first, treat macro-F1 as the primary success criterion, defend against leakage aggressively, and only then spend complexity budget on cross-modal transformers or graph models**. That is the shortest path to a project that is both publishable in spirit and defensible in engineering terms.

¹ ³ ⁷ ⁹ [f1_score — scikit-learn 1.8.0 documentation](#)

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html?utm_source=chatgpt.com

- 2 6 11 18 19 25 <https://aclanthology.org/P19-1050/>
<https://aclanthology.org/P19-1050/>
- 4 13 **Introducing Whisper**
https://openai.com/index/whisper/?utm_source=chatgpt.com
- 5 10 22 **ONNX Runtime | onnxruntime**
https://onnxruntime.ai/docs/?utm_source=chatgpt.com
- 8 **dbmdz/bert-base-turkish-cased**
https://huggingface.co/dbmdz/bert-base-turkish-cased?utm_source=chatgpt.com
- 12 14 **<https://librosa.org/doc/main/generated/librosa.feature.mfcc.html>**
<https://librosa.org/doc/main/generated/librosa.feature.mfcc.html>
- 15 **<https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>**
<https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>
- 16 17 **<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.VotingClassifier.html>**
<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.VotingClassifier.html>
- 20 **Get Started: Data Pipelines**
https://doc.dvc.org/start/data-pipelines/data-pipelines?utm_source=chatgpt.com
- 21 **StratifiedGroupKFold — scikit-learn 1.8.0 documentation**
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedGroupKFold.html?utm_source=chatgpt.com
- 23 **<https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>**
<https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>
- 24 **<https://aws.amazon.com/ec2/instance-types/g4/>**
<https://aws.amazon.com/ec2/instance-types/g4/>
- 26 **Transformers**
https://huggingface.co/docs/transformers/en/index?utm_source=chatgpt.com